



**POLITECNICO
MILANO 1863**

**Autonomous Vehicles - 055657
Mechanical Engineering
Automotive and Motorsport Engineering, M.Sc.**

Group:
Pietro Nardon

June 24, 2025

Contents

1	<u>Assignment 1: Input velocity computation</u>	1
1.1	Minimum distance with respect to obstacles	1
1.2	Input velocity command estimation	1
1.3	Accuracy evaluation	2
2	<u>Assignment 2: Feedback control</u>	4
2.1	Waypoints and constraints	4
2.2	Feedback control system	4
2.3	Results	5
3	<u>Assignment 3: Colour camera recognition</u>	7
3.1	Sphere definition	7
3.2	HSV Approach	7
3.3	Centroid evaluation	8
3.4	Feedback control loop	8
3.5	Results	9
4	<u>Assignment 5: Parking gate control</u>	10
4.1	Finite State Model design	10
4.1.1	Vehicle FSM	10
4.1.2	Gate FSM	11
4.1.3	Raising and Lowering functions	12
4.2	FSM simulation results	13
4.3	Comments	14
5	<u>Project 4: Four-way intersection</u>	15
5.1	Introduction	15
5.2	Finite-State Machine	16
5.3	Vehicle motion law model	16
5.3.1	Approaching the intersection	17
5.3.2	Waiting at the stopping line line	17
5.3.3	Turning	17
5.3.4	Straight direction	19
5.3.5	Leaving the intersection	20
5.3.6	Velocity profiles	20
5.4	Road rules implementation	21
5.4.1	Direction choice	21
5.4.2	Waiting state	21
5.4.3	Crossing conditions	22
5.4.4	Special intersection cases	23
5.5	Plotting mechanism	24
5.6	Results	25
5.6.1	Different number of vehicles	25
5.6.2	All vehicle going in the same direction	26
5.6.3	Stopping in the intersection	27

5.6.4	Random directions	28
5.7	Conclusions	29

1 Assignment 1: Input velocity computation

This assignment focused on retrieving data from a bag recorded while moving a Waffle Pi Turtlebot in the Gazebo environment using keyboard commands. From this bag various data and quantities could be studied and analysed.

1.1 Minimum distance with respect to obstacles

At first the minimum distance of the Turtlebot from any obstacle was analysed and plotted in (Fig. 1.1). In order to retrieve the necessary data to perform the calculation a subscription to the provided bag was carried out. In particular the data coming from the scanner were studied: the variable `Ranges` containing the distance from an obstacle, was evaluated at each time step, extracting the minimum value among all the values along all 360° , plotting it against time in (Fig. 1.1).

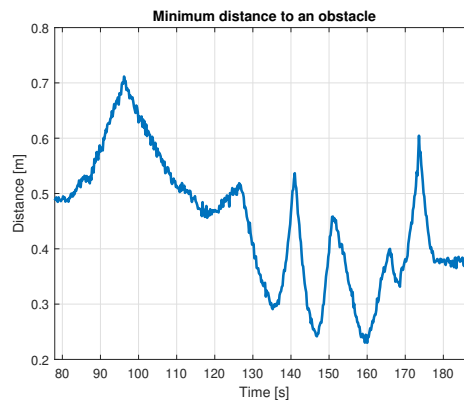
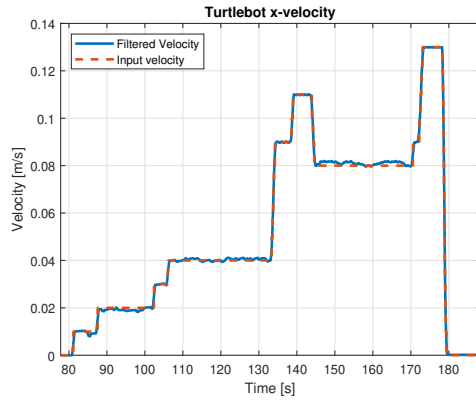


Figure 1.1: Obstacle minimum distance

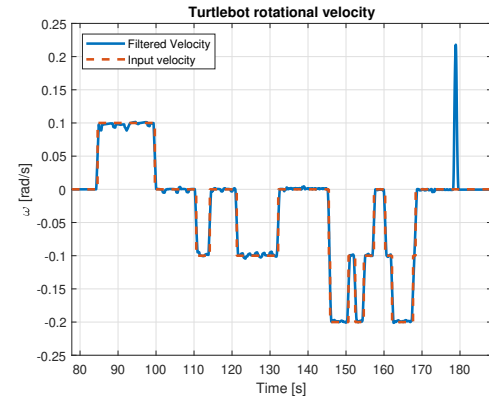
1.2 Input velocity command estimation

The part of the bag containing the odometry of the robot was used to estimate the input velocity imposed. In particular, the variable `Twist` inside the odometry was analysed: after extracting the data a moving mean filter was applied to clean the signal of noise, then it was rounded at its second decimals for the x velocity and at its first decimal for the rotational speed. This was made since, when using the `teleop` command in Gazebo, the translational velocity is imposed with a discretisation with two decimals (i.e. pressing once the key increases the speed by 0.01 m/s) while the rotation has a one decimal discretization (i.e. pressing once the key increases the rotational speed by 0.1 rad/s). The velocity along the y direction was not considered since it is not controlled directly with keyboard inputs, but it is only a consequence of the inputs given on x and on the rotation.

The results can be seen in (Fig. 1.2), a peak at around 178 s can be seen in the rotational velocity, since this happens while the robot is stopping it was assumed to be some kind of numerical error and not a real input from the keyboard, thus it was not included in the estimated input velocity, as visible in (Fig. 1.2b). The topic was then saved into a bag to be used multiple times. In order to publish the input to the robot a Simulink file was created, in which a `/cmd_vel` message was published, containing the two computed velocities.



(a) Translational velocity



(b) Rotational velocity

Figure 1.2: Velocity profiles

1.3 Accuracy evaluation

To evaluate how accurate the estimation of `cmd_vel` was, the estimated commands were published to the Turtlebot and its consequent behaviour was analysed.

The estimated commands were published to the Turtlebot using Simulink: a new `Twist` message was created, then, using a bus selector, the linear velocity along the x axis and the rotational velocity about the z axis were assigned and sent through with a publisher, as shown in (Fig. 1.3).

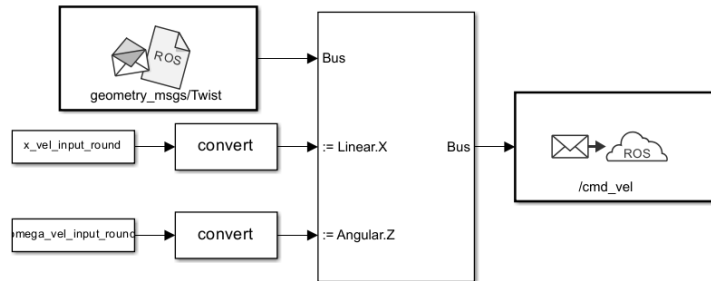


Figure 1.3: Simulink publisher

To analyse the accuracy of the estimated commands, the original trajectory was plotted against the one taken with the estimated command and are presented in (Fig. 1.4), moreover also the measured velocities were compared, since the original `cmd_vel` signal was not given. The measured velocity were analysed studying the `Twist` signal coming out of `odom`.

The trajectories are very close to each other up until the last turn, where they seem to diverge significantly respect to the rest of the path. This divergence is probably given by the input velocities being not perfectly synchronised, by looking at both cases in (Fig. 1.5) it is possible to notice that the estimated velocity slightly leads the original one, this is probably due to when the rounding and sampling took place. This leading can be seen in the beginning of the trajectory, where the robot makes a tighter turn, but also at the end of the trajectory, where the robot stops turning too early, even though the translational velocity is still very high. This small synchronisation error, made at a very high speed, results in a different orientation of the vehicle after the last tight turn, leading to a big divergence towards the end.

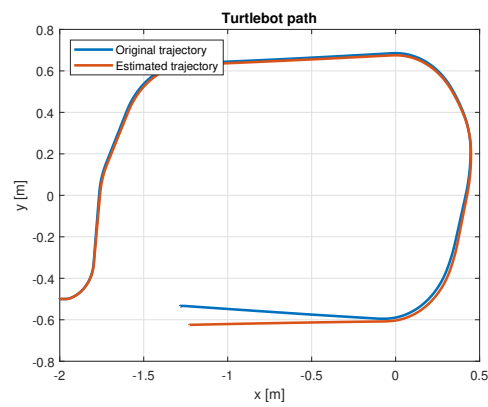
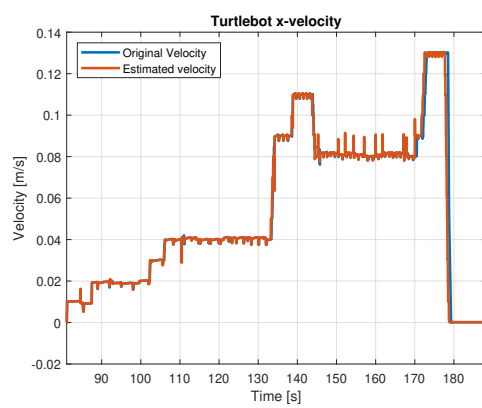
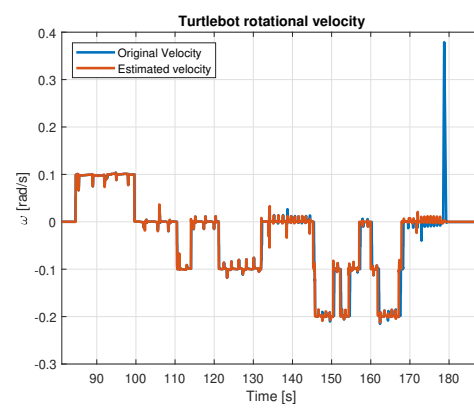


Figure 1.4: Trajectory



(a) Translational velocity



(b) Rotational velocity

Figure 1.5: Velocity profiles

2 Assignment 2: Feedback control

Objective of the assignment was to design a feedback control system for the Turtlebot. Starting from a list of waypoints provided, the control system should start the Turtlebot from the origin and move it to validate each waypoint, stopping at the last one.

2.1 Waypoints and constraints

The 10 waypoints that the Turtlebot had to validate are reported in (Tab. 2.1).

In order to validate a waypoint, the Turtlebot had to get at a distance of maximum 0.05 m from the waypoint, then it would fix its heading by rotating in place.

	1	2	3	4	5	6	7	8	9	10
x	5	5	-5	-5	0	3	-3	0	3	0
y	0	5	-5	5	0	3	0	-3	0	0
θ	1.57	3.92	1.57	0	0	2.35	4.71	0.78	1.57	4.71

Table 2.1: Waypoints

The physical constraints the control system had to satisfy were:

- $|v_{max}| = 0.2\text{ m/s}$
- $|\omega_{max}| = 0.4\text{ rad/s}$

2.2 Feedback control system

The Simulink feedback system (Fig. 2.1) consisted in subscribing to the Turtlebot's position using /odom, then the difference between the /odom output and the selected waypoint was computed. This difference served as input for the control system.

The control system was effectively differentiated depending on the scenario: when the robot was far from the waypoint, the control commands would be given in a certain way, while when the robot had validated a waypoint the commands would change. A counter was implemented to store when a waypoint had been validated, to then move to the next one.

- Distance from WP $> 0.05\text{ m}$: when the robot was far from the waypoint, the input velocity was computed proportionally with the position error. The longitudinal velocity was proportional to the distance multiplied by a gain $K_p^v = 0.2$, saturated at $v_{max} = 0.2\text{ m/s}$.
The rotational velocity was proportional to the rotation needed to reach the waypoint's x and y coordinates (not the angle θ), with a proportional gain $K_p^\omega = 2$, saturated at $\omega_{max} = \pm 0.4\text{ rad/s}$. In this way the robot could reach the waypoint's position quickly.
- Distance from WP $\leq 0.05\text{ m}$: when the robot was close enough to the waypoint, the Matlab function `drive` would operate the controls of the robot. Once arrived at the waypoint's coordinates, it would slow down and rotate on place to align with the waypoint's angular position θ , imposing a rotational velocity proportional to the error with respect to θ , with $K_p^\omega = 0.7$, saturated at $\omega_{max} = \pm 0.4\text{ rad/s}$. Once the waypoint was validated, it would start to get away from it, starting by rotating in place in order to position itself aiming to the next waypoint.

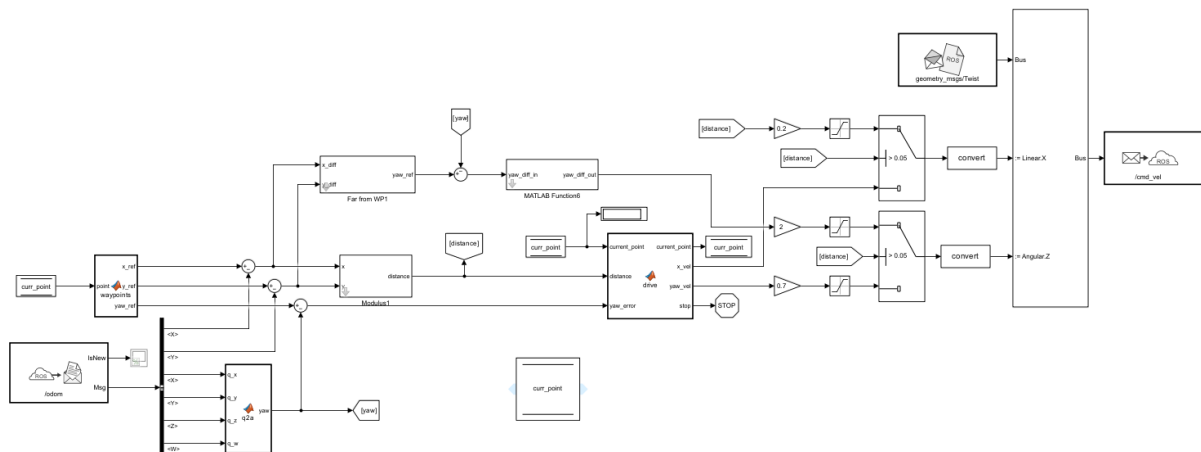


Figure 2.1: Feedback control system

Since the current waypoint that the Turtlebot had to aim was inserted in a loop in which the point choice depended on the point itself, it was needed to store a counter outside of the loop, using the so called Data Store Memory. In this way the variable determining the current waypoint was the same for each block in the same time-frame of the simulation.

Inside the Matlab function drive a small if condition was also present to stop the entire simulation once the last waypoint was reached: the simulation was stopped with the Turtlebot positioned in a stand still at the last waypoint.

2.3 Results

In (Fig. 2.2) the resulting trajectory of the robot is presented (in blue), with the numbered waypoints (in red). Here the control system behaviour described before is visible, as the Turtlebot at first aims to the new waypoint rotating on place, it drives towards it in a straight line, then when it is inside the waypoint it rotates to the desired θ in order to validate it.

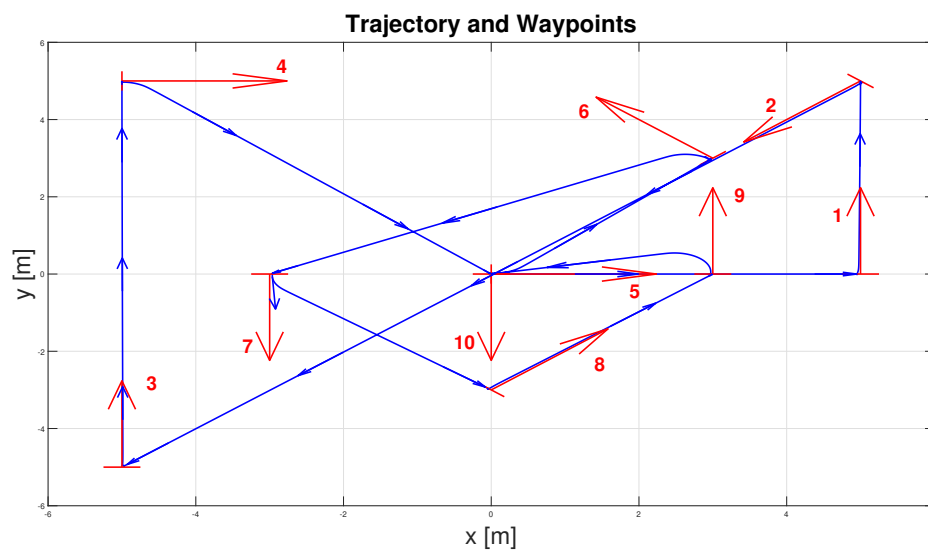


Figure 2.2: Trajectory and Waypoints

The linear velocity command is plotted in (Fig. 2.3), where the 10 "valleys" represent the 10 waypoints, where the Turtlebot has to stop, leading its velocity to drop to 0 m/s . The velocity is maintained at 0 m/s for a short period of time before departing again to allow the Turtlebot to "aim" the new waypoint. Moreover, it is also possible to study the proportional control system's behaviours, as when the robot is approaching the waypoint, its velocity starts to drop. While it is cruising towards the new waypoint its imposed speed is fixed at 0.2 m/s .

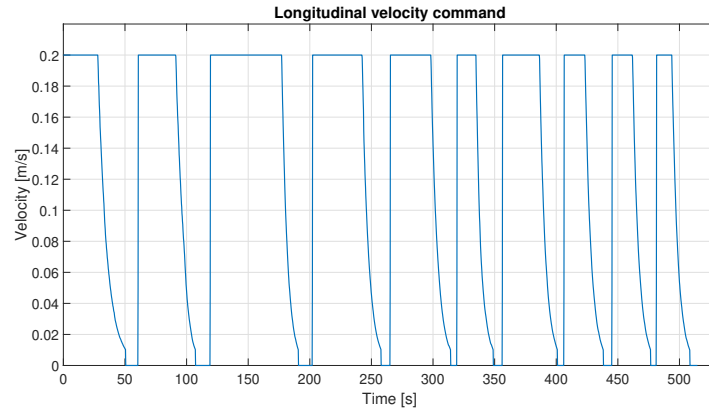


Figure 2.3: Linear Velocity

The imposed angular speed is proposed in (Fig. 2.4) instead.

Two peaks, very close to each other, are present when the robot is inside a waypoint. The first peak is due to the Turtlebot correcting its alignment in order to validate a waypoint. Instead, the second peak shows up when the Turtlebot starts moving again, this rotation is used to effectively "aim" the new waypoint. Some small oscillations are present when approaching a waypoint, this derives from the nature of the controller itself: since it is also proportional to the rotation error between the Turtlebot actual position and the waypoint position, it is natural that as the distance decreases it would need to make more adjustments. This could be partially mitigated by decreasing K_p^ω .

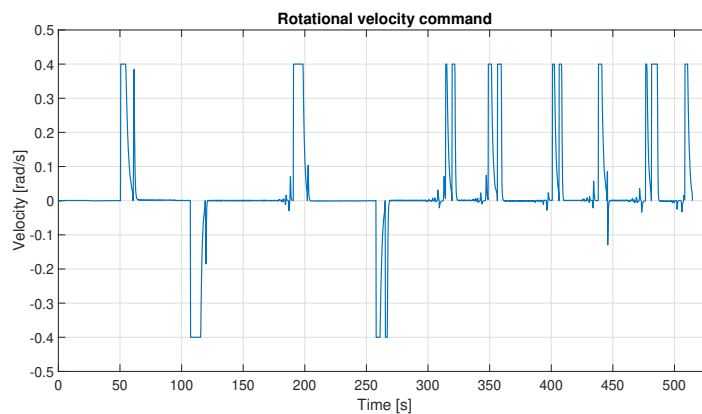


Figure 2.4: Angular Velocity

3 Assignment 3: Colour camera recognition

The aim of this assignment was to implement a feedback control system capable of moving the Turtlebot towards an object of a chosen colour, and crash into it. The main challenge was the processing of the images collected by the camera, in order to find the aimed object and compute its position respect to the Turtlebot. Several image processing approaches based on colours were considered, and the best one was found to be the HSV one, later described in (Sec. 3.2).

The request was for the Turtlebot to look for a red sphere in the world, once the Turtlebot had found it, it would aim and move towards it, updating its heading in the process, in order to crash into it.

3.1 Sphere definition

To set properly the Gazebo empty world, a provided code was used to define two spheres:

- A red sphere (`#c83030`), the target, with radius of 0.1 m , in position $[0, 6]$.
- A purple sphere (`#c80067`), with radius of 0.2 m , in position $[6, 0]$.

This resulted in a setup of Gazebo as shown in (Fig. 3.1).

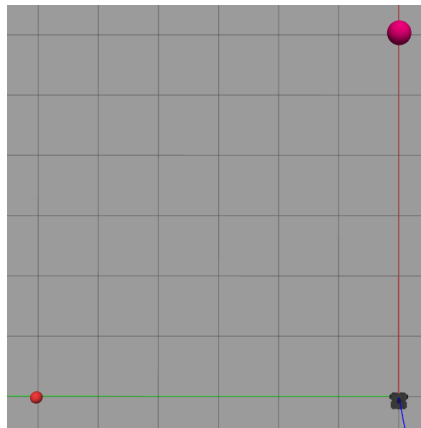


Figure 3.1: Gazebo world

3.2 HSV Approach

This image processing approach works similar to a human eye: is based on 3 parameters that together are able to detect a chosen colour even in not favourable light conditions. The three parameters are:

- *Hue*: it represents the actual colour of an object, defined from 0 to 360.
- *Saturation*: describes the amount of gray in the colour of an object, defined from 0 to 1.
- *Value*: describes the brightness of the colour of an object, defined from 0 to 1.

To detect a colour with this approach, a combination of these three parameters is needed. In particular, a range, for each parameter, is chosen, in which to look for the desired colour: by carefully tuning these ranges is possible to detect any type of colour.

The chosen parameters are collected in (Tab. 3.1), where *Value* was left the default one.

	Up	Low
Hue	0.3	0
Sat	1	0.7

Table 3.1: HSV setup

3.3 Centroid evaluation

After the desired sphere was located, a method to define its position was needed. Therefore, a blob analysis was carried out, in this way it was possible to locate the sphere's centroid. The centroid's coordinates, measured in pixels, were fed to the feedback control loop moving the Turtlebot.

In (Fig. 3.2) *BW* is the output of the HSV analysis, while the *Matlab* function is used to select the sphere with the closest centroid in case more spheres with the same colour are present.

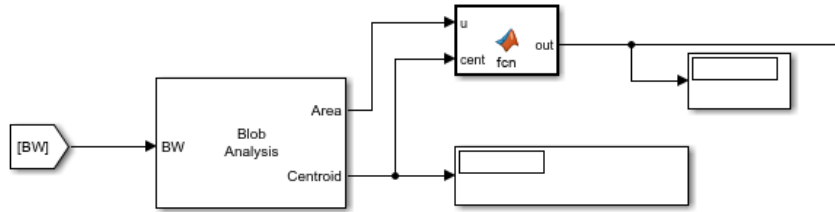


Figure 3.2: Blob analysis Simulink

3.4 Feedback control loop

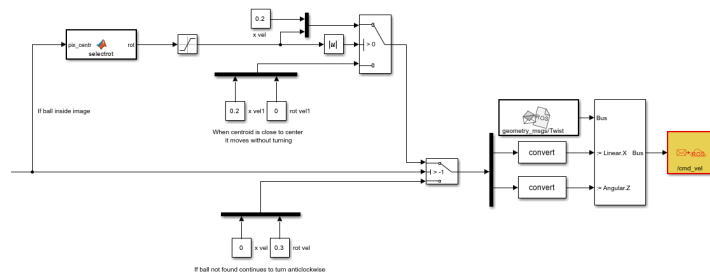
A proportional feedback control loop was developed to move the Turtlebot and drive it towards the selected sphere, it is shown in (Fig. 3.3). The input of the loop is the centroid position of the closest selected ball, computed using the Simulink blocks in (Fig. 3.2).

The loop is divided in two sections:

- *Searching section:* this section is selected when the centroid of the ball is not visible by the camera, therefore a rotational velocity equal to 0.3 rad/s is imposed. This section is exited when the ball's centroid is visible by the camera.
- *Moving section:* when the ball's centroid is visible by the camera, the *Matlab* function `selectrot` computes the rotational velocity to impose as $\omega = K_p(320 - pos_{centr})$ where $K_p = 4 \cdot 10^{-3}$. The output value was then saturated up to the maximum rotational velocity that the Turtlebot can achieve.

The imposed linear velocity will be $v = 0.2 \text{ m/s}$ and the rotational one will be ω .

In both cases the velocity will be converted and sent as a output message to the Turtlebot as `/cmd_vel` messages.



3.5 Results

As visible from the plots below, the robot, controlled by the designed controller, is able to fulfil its requirement of crashing against the red sphere. By analysing (Fig. 3.4), is clear that the robot crashed into the sphere as its final position coincides with the original position of the sphere described in (Sec. 3.1). In (Fig. 3.5) is shown that the robot was able to maintain its trajectory keeping the centroid's position in the middle of the camera, which is equal to 320 pixel , up until it crashed against it, at around 33 s . In (Fig. 3.7) the angular velocity imposed to the robot is shown. Here can be seen how at the beginning of the simulation the robot started looking for the sphere, rotating at higher angular speeds. Once it had found it, the angular velocity drops to 0 rad/s , with very small adjustments to keep the correct trajectory.

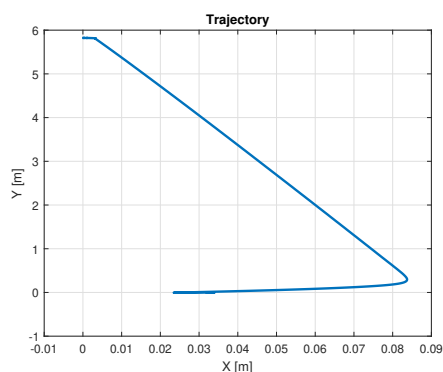


Figure 3.4: Trajectory

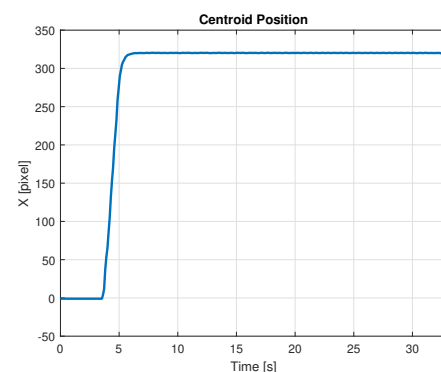


Figure 3.5: Centroid position

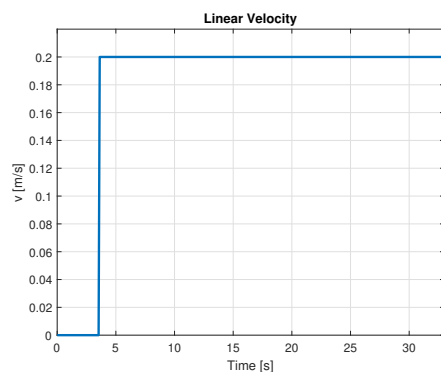


Figure 3.6: Linear Velocity

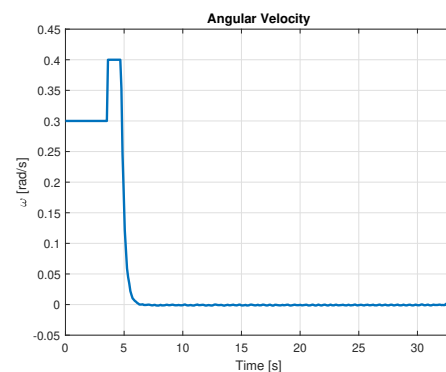


Figure 3.7: Angular Velocity

4 Assignment 5: Parking gate control

Scope of the assignment was to create a controller for a parking gate, using Finite State Modelling logics and architecture, inside Simulink. A vehicle would start at a random position and travel towards the gate, when at the gate it would stop and wait for the gate to open, after the gate had completely opened the vehicle resumed travelling. The gate would then closed after the vehicle had passed.



Some constraints of the gate position, speed and acceleration were present, to simulate a real environment.

Positions		Dynamics	
<i>Down</i>	0°	v_{max}	4 deg/s
<i>Up</i>	90°	a_{max}	$ 1 \text{ deg/s}^2$

Table 4.1: Gate constraints

4.1 Finite State Model design

The FSM consisted in two main parts: the vehicle, already provided, and the actual gate control, created from scratch. They were run in a parallel setup, since both models needed to run at the same time, and the outputs of one determined the actions of the other and vice versa.

4.1.1 Vehicle FSM

The starting distance of the vehicle from the gate was set randomly, with a negative value, while the gate position was defined as 0. When the vehicle was manually started, it would approach the gate at a speed of $v = 1 \text{ m/s}$ and, when it reached a position greater or equal than 0, meant it had arrived at the gate. Once at the gate, it would stop and wait until the gate opened completely, then, it would resume to travel.

A second vehicle would start again at the random position after 10s had passed when the first vehicle exited the gate.

4.1.2 Gate FSM

The gate FSM was created from scratch and run in parallel with the vehicle FSM, as they both needed variables coming from the other model, such as signals showing that: the vehicle is waiting, the gate has opened, the vehicle has passed, the gate is closed. Moreover, a logic was also implemented, to act as a sensor detecting when a vehicle is in front of the gate while in the lowering state.

The architecture of the FSM can be described as follows:

- After the simulation is launched the gate enters an idle state, in which the control variables `angle` and `v_actual` are defined, and a variable `gate_up` defining whether the gate is closed or open.
- When a vehicle has approached the gate and it is waiting for it to open, the state changes and the gate starts to raise based on the function `raising`, described in (Sec. 4.1.3). Inside this state, the difference between time instants is computed to be used as input for `raising`.
- After the bar has reached the *Up* position the state changes into a waiting state, where is signalled that the gate is up, the bar is kept at the open position. Only after a vehicle has passed this state is exited.
- Once a vehicle has passed the bar starts to lower and is signalled that the gate is now closing. The lowering is made in a similar fashion to the raise phase, with just a change of sign when computing the angle inside `lowering`. If a car is waiting in front of the gate while it is closing then the state is changed back to raise it again, returning back to the raise state.
- When the bar is at the *Down* position the state is changed into closed, where is signalled that the gate is now completely closed, after this the gate is once again set in the idle state.

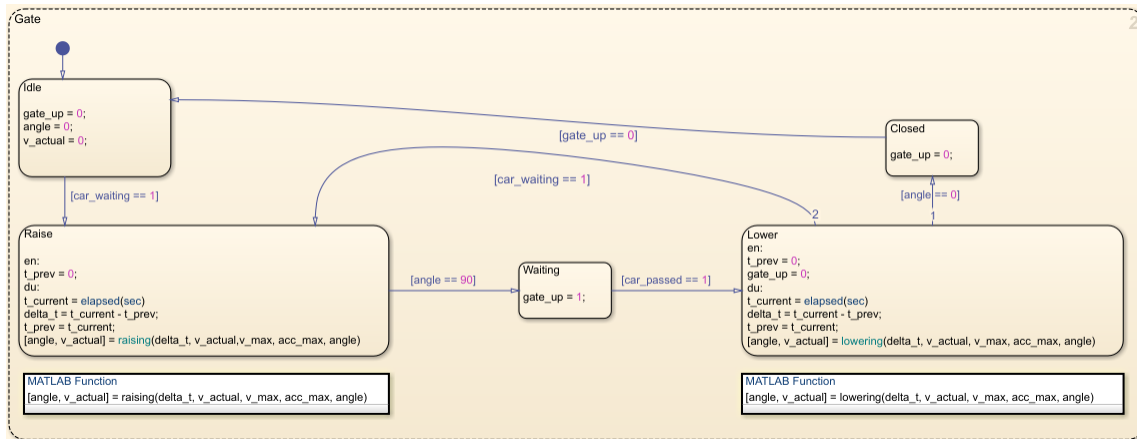


Figure 4.1: Gate FSM in Simulink

Since the gate closing procedure is slow, the second vehicle approaches the gate just after it completely closed. Since the initial position of the vehicle is random, it could happen that a vehicle arrives at the gate while it is closing. If a logic that solves this problem is not implemented, there is the risk that a vehicle approaches the gate while it is closing. In this case the vehicle has to wait for the gate to close, notice that a car is waiting and open again, increasing the waiting time in front of the gate. This would be a useless waste of time, which can also be frustrating for the vehicle's driver.

4.1.3 Raising and Lowering functions

Both raising and lowering were designed in a similar fashion. In order to compute the gate velocity and angle at a particular time instant, the uniformly accelerated motion laws (Eq. 4.1) were applied.

$$\begin{cases} v = v_0 + a\Delta t \\ \alpha = \alpha_0 \pm v\Delta t \end{cases} \quad (4.1)$$

The angle computation is different between the two phases. When raising the initial position is 0° and the angle increases up to 90° following the computed v_{actual} , when lowering the opposite happens, hence the $\pm$ sign in (Eq. 4.1).

Both functions started with null initial velocity, the velocity was selected using a proportional controller, in order to accelerate up to v_{max} and then slow down when approaching the target angle. A check on whether the gate should accelerate or decelerate was carried out, and then the velocity chosen was the one respecting the constraints.

This was implemented inside the FSM using a Matlab function in both cases, here only the raising one is presented. When the target angle was reached, the gate stopped by imposing v_{actual} to be null. The only differences between raising and lowering were the sign inside the angle computation and the target angle used.

```
function [angle, v_actual] = raising(delta_t, v_actual, v_max,
    acc_max, angle)
    % Computing angle difference
    delta_angle = 90 - angle;

    % Desired speed
    kp = 0.5;
    v_des = kp*delta_angle;
    v_des = min(v_des, v_max);
    % Acceleration
    if v_actual < v_des
        v_actual = v_actual + acc_max*delta_t;
        v_actual = min(v_actual, v_des);
    % Deceleration
    else
        v_actual = v_actual - acc_max*delta_t;
        v_actual = max(v_actual, v_des);
    end

    % Angle update
    angle = angle + v_actual*delta_t;
    if angle >= 90 % Limit angle
        angle = 90;
        v_actual = 0;
    end
end
```

4.2 FSM simulation results

In (Fig. 4.2) the trajectories of both the vehicle and the gate were plotted in time. In this simulation the sensor detecting if a second vehicle is approaching while the gate is closing is activated. In the time span of $100s$ two vehicles approach the gate, in both cases the gate opens, the first time at $38.6s$ and the second at $94.4s$ and both vehicles pass through.

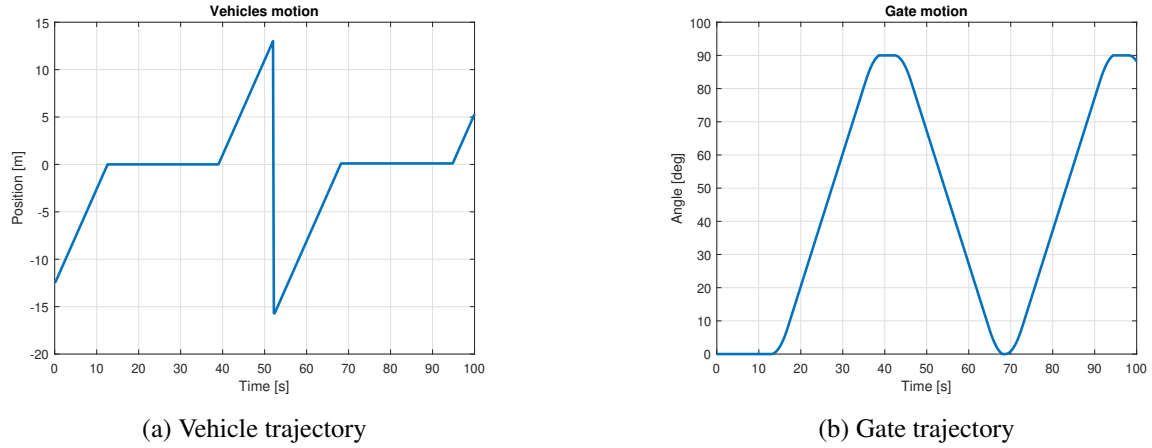


Figure 4.2: Vehicle and gate trajectories

By analysing (Fig. 4.2b) it is possible to notice that the second vehicle approaches the gate just after it completely closed: at around $68s$ the gate position is equal to 0° for a short period of time. This is also confirmed by (Fig. 4.2a), where the time spent by the two vehicles waiting for the gate to open is almost the same: $26.4s$ for the first vehicle and $26.6s$ for the second. This small discrepancy is probably due to numerical errors and the random starting position.

Also (Fig. 4.3) contains important informations regarding the gate velocity magnitude. The first and third square peaks correspond to the raising, while the second one to the lowering. Consequently the first and third valleys correspond to the gate waiting to close while in *Up* position, while the second valley corresponds to the idle state. Both during the raising and lowering phase the gate accelerates up to v_{max} while respecting the constraint of the acceleration.

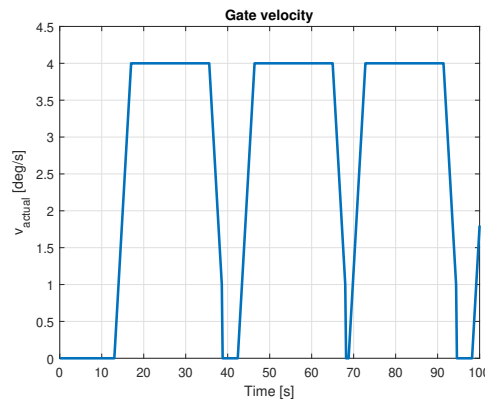


Figure 4.3: Gate velocity

4.3 Comments

The algorithm, even if simplified, can be used as a good starting point for the implementation of a real life controller.

Improvements can be made on the simulation of the mechanical system, in order to implement also non-linearities coming from the gate motor or from attrition, inertias, incorrect position of the gate and positioning errors around the *Up* and *Down* positions and more. Moreover the control logic, should be extended also to accommodate and consider the presence of the motor, which needs a control of its own.

To speed up operations, a pressure or magnetic sensor could be place before the gate to sense when a vehicle is approaching, but before it comes to a complete stop. In this way the raising state could be initiated a few seconds before, decreasing the waiting time.

5 Project 4: Four-way intersection

The final project chosen was based on the creation of a Finite-State Machine (FSM), to represent a four-way intersection scenario in which the involved cars had to respect the rules of the road.

5.1 Introduction

The vehicles would appear 20m before the intersection at full speed, then they would start to decelerate approaching the intersection, stopping before it. Here they would decide a random direction to travel (left, straight, right) and then, if the right of the vehicles was free, they would move to complete the manoeuvre. Both the velocity profiles and the road rules were managed inside the FSM, using simple point mass models for the first one, and a series of flags for the second, to pass from one state to the other.

The initial configuration, to be used as a reference, with all four vehicles, is presented, it was assumed for the roads to be perpendicular to each other. The road is 12m wide and all the vehicles travel in the middle of their lane, so at 3m from the right end of the road, the stopping line is positioned 6m from the centre.

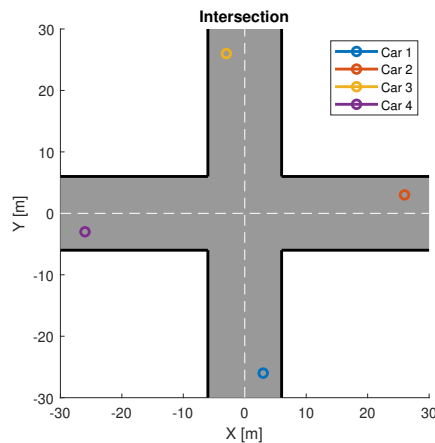


Figure 5.1: Initial configuration

A Simulink model, called `Project_Nardon`, contains all the necessary tools to implement the intersection logic. Inside of it lays a FSM model for each car, with a dedicated function to translate the motion carried out by each car into x and y coordinates to later plot the results.

A main code called `Project_Nardon_MAIN` activates the Simulink model and plots the results with a time-frame of 1s, to simulated a realistic scenario.

Inside the main code it is also possible to choose how many car will appear at the intersection by changing a boolean flag variable (for example, by setting `Start_2 = 1` the second car will appear). It is possible to select from 1 to 4 cars to drive simultaneously towards the intersection.

The vehicle physical limits are:

- Maximum acceleration and deceleration(with constant deceleration profiles): $a_{max} = |2| m/s^2$
- Maximum speed: $v_{max} = 30 km/h$
- Initial distance from stopping line: $pos_{in} = 20m$

5.2 Finite-State Machine

To implement correctly all the necessary motion law of the vehicles and the rules of the road, a FSM model was set up in Simulink. The FSM model contains four subsystems, one for each vehicle considered, that run all together in parallel. It receives a total of eight inputs, two for each vehicle: a random integer representing the direction of travel, and the boolean flag to activate/deactivate a vehicle. The model outputs a total of thirty-six outputs, eight for each vehicle: four are the actual values to plot representing the vehicle position, four velocities and four are boolean flags to signal the vehicle state.

Each car, as default transition, enters the *Idle* state, in which its position, initial speed and travel direction are initialized. Depending on the flag *Start* for the selected vehicle the FSM can enter either the *No_Start* state, where the variables controlling the road rules implementations are set to represent that the car is not present at the intersection, or the *Approaching* state, in which the car starts to slow down to approach the intersection.

5.3 Vehicle motion law model

The vehicles appear with a speed equal to v_{max} and at a fixed distance of pos_{in} from the stopping line, which is placed $6m$ from the centre of the intersection, therefore in (Fig. 5.1) the cars are placed $26m$ from the centre. The motion law was divided in three distinct sections, depending on which state each vehicle is in: approaching the intersection, crossing the intersection, leaving the intersection.

All states controlling the vehicle's motion have the same structure, and all the variables inside it are specific for the vehicle:

- **Entrance:** some flag variables representing the vehicle state are updated to signal the entrance in the state, putting them equal to 1. At the same time other variables needed to compute the motion are initialized, such as the velocity, angle of the turn and time.
- **During:** the time difference Δt between two instants of the simulation is computed in a loop, which is then fed to a Matlab function, specific for each direction, together with other inputs, to compute the instantaneous velocity and position along the intersection. Furthermore, inside all Matlab function is present a check to see if the vehicle's physical limits are respected.
- **Exiting:** updates of flags representing the vehicle state and the vehicle position inside the intersection and respect to other vehicles. Moreover, also an update on just the velocity is carried out, to smoothen the velocity profile transition when changing state.

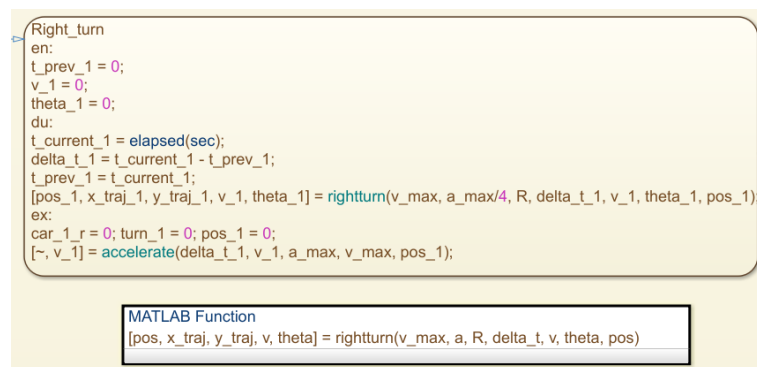


Figure 5.2: Example of motion state

5.3.1 Approaching the intersection

When the car enters the *Approaching* state, the flag variable *appr* is updated, then it enters the loop to compute Δt which is fed to the function *approach*, computing the instantaneous quantities of velocity and position. This state is then exited (and the flag variable updated) when the position variable *pos* is equal to 0, meaning that the vehicle is at the stopping line.

The *approach* function computes the vehicle's velocity and the position, simulating a smooth slowing down process happening when approaching the intersection. As inputs it takes the time difference Δt , the velocity and position at the current time step v and pos , the maximum achievable deceleration a_{max} ; together with the initial velocity and position v_{in} and pos_{in} .

Inside *approach*, the constant deceleration required to slow down from the initial position to the stopping line, when $v_{in} = v_{max}$, is computed.

$$a_{req} = -\frac{v_{in}^2}{2 \cdot pos_{in}} = -1.736 m/s^2 \quad (5.1)$$

With the acceleration a_{req} , the speed is updated.

$$v_{new} = v_{old} + a_{req} \cdot \Delta t \quad (5.2)$$

Finally, the updated position is calculated by using the average speed between the old and new speed computed inside a time-step. Due to the large time-step of 1 s (which is used by the FSM inside Simulink) it was decided to use the average speed to capture better the change of speed inside a time-step, allowing for the car to correctly stop at the stopping line.

$$pos_{new} = pos_{old} - v_{avg} \cdot \Delta t \quad (5.3)$$

5.3.2 Waiting at the stopping line

When the *Approaching* state is exited, the FSM enters the *Waiting* state. Here, a flag variable is updated and the vehicle speed is set again to 0 m/s; then, a new state between *Left*, *Right* and *Straight* is selected depending on the random direction the vehicle has to travel. In these states, a variable signalling the intention of the vehicle of entering the intersection and one storing the direction, are updated; subsequently, when the state is exited, based on the road rules explained in (Sec. 5.4), flags signal the finish of the *Waiting* state and the beginning of the turning phase.

5.3.3 Turning

All functions concerning a turn are structured in the same way, with some minor changes differentiating them. They all use a variable θ representing an angle to keep track of where they are along the curve. To simplify the calculations the trajectory was assumed to be a circular path of radius R (as a default it was set to $R = 3 m$, but it can change depending on the direction), travelling for 90° along the path.

Inside the different functions the x and y coordinates were computed to be used in the plots, as well as the velocity v and the position travelled pos . To exit the state it was necessary to have completed the turn, meaning that the travelled distance along the path had to be $pos = R \frac{\pi}{2}$. All these functions take as inputs the actual velocity v , position pos , time-step Δt and angle θ , together with the turning radius R and the desired acceleration a and the maximum velocity v_{max} .

The velocity was updated using the simple motion law, using different accelerations depending on the direction, then the angle θ was computed using the average velocity v_{avg} in the time step. The signs of the equations depend on the vehicle's intended direction.

$$\begin{cases} v_{new} = v_{old} + a\Delta t \\ v_{avg} = \frac{v_{new} + v_{old}}{2} \\ \theta_{new} = \theta_{old} \pm \frac{v_{avg}}{R} \cdot \Delta t \end{cases} \quad (5.4)$$

Knowing θ_{new} it was possible to compute the x and y coordinates.

$$\begin{cases} x = R(1 - \cos(\theta_{new})) \\ y = \pm R \sin(\theta_{new}) \end{cases} \quad (5.5)$$

The position instead was computed using the speed at the end of the time step.

$$pos_{new} = pos_{old} + v_{new} \cdot \Delta t \quad (5.6)$$

Right turn When the direction the vehicle has to travel is towards the right, the state *Right turn* is selected. Inside the Matlab function `rightturn` (Eq. 5.4) and (Eq. 5.5) are used both with the minus sign, together with (Eq. 5.6). Since a vehicle turning to the right has to remain on the right lane after the intersection, the turning radius was set to $R = 3m$, therefore the state was terminated when $pos = R \frac{\pi}{2} = 4.71m$; the acceleration was set lower than the maximum the vehicle could achieve to $a = \frac{a_{max}}{4} = 0.5m/s^2$, due to the tight turn.

Upon exiting the flags are updated and the velocity is updated one more time.

Left turn If the travel direction of the vehicle is towards the left, then the state *Left turn* is selected. Inside the Matlab function `leftturn` (Eq. 5.4) and (Eq. 5.5) are used both with the plus sign, together with (Eq. 5.6). To take the correct lane the vehicle has to perform a $9m$ radius turn, therefore the value used as input in the functions is $3R$, resulting in exiting the state at $pos = 3 \cdot R \frac{\pi}{2} = 14.13m$; the acceleration instead was set equal to $a = \frac{3}{4}a_{max} = 1.5m/s^2$ since the curve is longer.

Also in *Left turn*, upon exiting, the flags are updated and the velocity is updated one more time.

Occupying the intersection In the particular case when the vehicle has to occupy the intersection while going in the left direction, explained in (Sec. 5.4), a modified version of the *Left turn* state is used. Since the car had to stop after running for 45° inside the intersection, the function `leftturn` was modified to tackle also the stopping part, resulting in the creation of `leftturn45`.

A logic was introduced to decide whether to keep on accelerating or start decelerating to stop in the intersection, using the distance remaining. The total distance to travel in this manoeuvre was equal to $arc_{tot} = 3R \frac{\pi}{4}$, if the distance remaining was shorter than the distance needed to stop starting at that particular speed, the vehicle would start to brake. The updated vehicle speed v_{new} would have the plus or minus sign depending on the action taken, where $a = \frac{a_{max}}{2} = 1m/s$.

$$\begin{cases} arc_{rem} = arc_{tot} - pos \\ d_{stop} = \frac{v^2}{2a} \\ v_{new} = v_{old} \pm a \cdot \Delta t \end{cases} \quad (5.7)$$

After updating the speed the function `leftturn45` is equal to the one used for the *Left turn*.

Part of the leftturn45 function.

```
% Remaining arc length
arc_remaining = (pi/4)*R - pos;

% Compute stopping distance from current speed
d_stop = v^2/(2*a);

% Decide whether to brake or accelerate
if arc_remaining <= d_stop
    v_new = max(v - a*delta_t, 0);          % Brake
else
    v_new = min(v + a*delta_t, v_max);      % Accelerate
end

% Average velocity for position update
v_avg = (v + v_new)/2;

% Update angular position
theta = theta + (v_avg/R)*delta_t;

% Plotting coordinates for left turn
x_traj = R*(1 - cos(theta));    y_traj = R*sin(theta);

% Update arc length
pos = min(arc_total, pos + v_avg*delta_t);

% Update output velocity
v = v_new;
```

5.3.4 Straight direction

When the direction selected is straight, the FSM enters the *Straight_turn* state. Upon entrance, time and velocity variables are initialised to 0, then the function `straightturn` computes both velocity v and position pos . Inside `straightturn` a very simple motion law is used to compute the outputs, here it was decided to use as acceleration the maximum achievable one a_{max} , to free faster the intersection. The state is exited when the car travelled for a distance equal to $pos = 4R = 12m$. Upon exiting flags are updated and the velocity is calculated using the function `accelerate` to leave the intersection.

$$\begin{cases} v_{new} = v_{old} + a_{max} \cdot \Delta t \\ pos_{new} = pos_{old} + v_{new} \cdot \Delta t \end{cases} \quad (5.8)$$

5.3.5 Leaving the intersection

All the states computing the motion inside the intersection, when exited, lead to the last state, responsible for computing the acceleration phase when leaving the intersection to return at cruising speed. When the state *Leaving* is entered, and the appropriate flags are updated. Here, like the other states, the time step Δt is computed and fed to the Matlab function `accelerate`. This function computes the position and velocity of the vehicle, after it has left the intersection, using the same law of motion of (Eq. 5.8), again using the maximum achievable acceleration a_{max} . Inside the function is present also a check to limit the velocity to v_{max} .

5.3.6 Velocity profiles

In (Fig. 5.3) all the possible velocity profiles are plotted. The manoeuvres performed are:

- **Car 1:** Half left turn, occupying the intersection, freeing the passage for other cars.
- **Car 2:** Left turn after Car 4 passed the intersection.
- **Car 3:** Right turn just after it arrived at the intersection.
- **Car 4:** Straight crossing after Car 1 freed the way.

In all cars performing a turn (meaning 1,2,3), towards the end of the simulation, is possible to appreciate the change in acceleration when the *Turning* state is abandoned in favour of the *Leaving* one; nevertheless the acceleration profile is always constant inside a state, albeit it can be different in different states.

Both at the beginning and at the end of the simulation, the speed achieved is equal to v_{max} .

The velocity profile of Car 1 shows the manoeuvre made to occupy the intersection.

In each possible combination of travel directions all the velocity profile will be equal to those in (Fig. 5.3), but possibly with different waiting times.

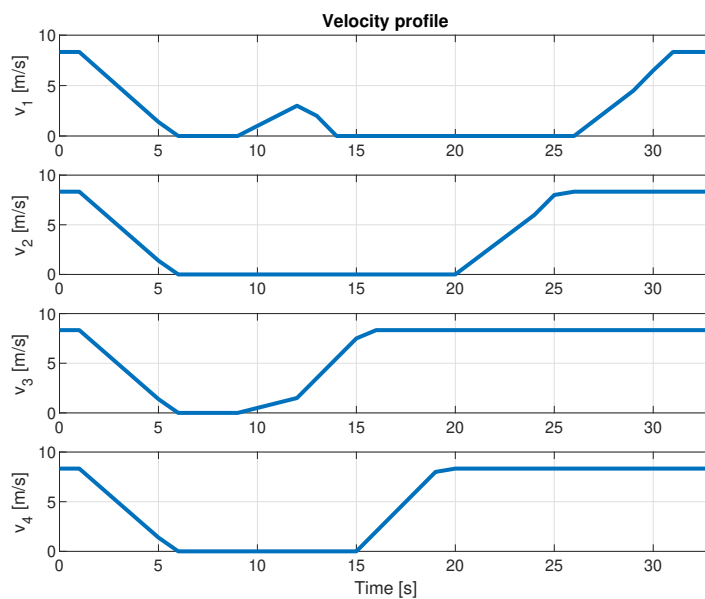


Figure 5.3: Different velocities profiles

5.4 Road rules implementation

In a four-way intersection a vehicle has three different possible directions: going left, right or straight. Depending on the chosen direction, and on the vehicles occupying the intersection, the vehicle may have to remain still and wait for other vehicles to leave the intersection before occupying it on its own.

The logic deciding the actions to take while waiting to cross the intersection is contained inside each FSM representing the single vehicle. Several FSM states, positioned before the motion states, decide in which state to enter and more importantly when to enter a state.

5.4.1 Direction choice

As default, the direction in which the vehicles has to travel is chosen randomly each time the simulation is run. This was implemented by using the `Random integer` block inside Simulink, which can output three different numbers, each representing one direction, and it is taken as input by the FSM under the variable `Dir_x`, where `x` is the vehicle number. The directions are coded as follows:

- `Dir_x = 0`: the vehicle `x` will turn left.
- `Dir_x = 1`: the vehicle `x` will go straight.
- `Dir_x = 2`: the vehicle `x` will turn right.

It is also eventually possible to pass into a "manual" choice of direction, by actuating a switch inside `Project_Nardon`, to choose the direction manually by changing the value of a `Constant` block.

5.4.2 Waiting state

After the vehicle has stopped at the waiting line, as described in (Sec. 5.3.1), the vehicle enters the *Waiting* state. Upon entering the state, the variable `wait_x` is set equal to 1, signalling that the vehicle is at the intersection, waiting to begin the manoeuvre. After the flag is set, the state is changed, depending on the travel direction.

In the new state a variable `car_x_r` is set to 1, to tell other vehicles that the vehicle `x` is going to occupy the intersection and that it needs to be given right of way; the variable `priority_x` instead stores the priority the vehicle has to occupy the intersection, being going right the highest priority, left the lowest.

To exit the state, and consequently start moving inside the intersection, a series of conditions must be satisfied (Sec. 5.4.3), each suited to the different travel direction. Upon exiting the state, flags showing that the vehicle is not waiting any more but it is starting to move, are updated.

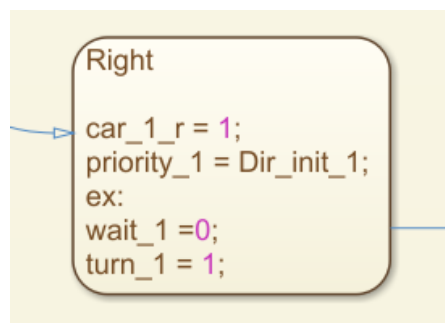


Figure 5.4: Example of waiting state before turning right

5.4.3 Crossing conditions

Following the road rules, three different conditions, depending on the travel direction, were created:

- **Right direction:** the vehicle has the right of way, if other vehicles, following other directions, have to go in the same lane, they must give way to the right turning vehicle. This situation was implemented simply by not putting any condition to pass from the waiting state to the turning one, so a vehicle turning right will simply stop at the intersection and then resume to travel directly.
- **Straight direction:** a vehicle going straight has to check if the vehicle on its right (if present), is just occupying the intersection, whatever the travel direction, and give it way. The implementation was carried out by using the flag `car_x_r`: the vehicle would wait as long as it is equal to 1, when its value changes to 0 means that the vehicle on the right has finished the manoeuvre, so the right is free, therefore it can start travelling.
- **Left direction:** when turning left it is necessary to give way both to the vehicle on the right (unless it is going right), as well as the vehicle on the left (unless it is going left as well). It was assumed the street to be wide enough for the vehicles to pass without crashing, even in the case of both vehicles turning left. This condition, probably the most complex one, was implemented with a `and` and `or` logic using both the `car_x_r` and `priority_x` variables.

In (Tab. 5.1) is reported an example using Car 3 to clarify this problem. The vehicle on the right is always the vehicle bearing the index following the studied vehicle (so for Car 3 the vehicle on the right is Car 4). The vehicle in front is therefore, going anti-clockwise, the one whose index is +2 the index of the vehicle considered (so for Car 3 the vehicle in front is Car 1). Only if the following conditions are met the vehicle can begin the manoeuvre.

Direction	Condition
Right	-
Straight	<code>car_4_r == 0</code>
Left	<code>(car_4_r == 0 priority_4 == 2) && (car_1_r == 0 priority_1 == 0)</code>

Table 5.1: Car 3 crossing conditions

5.4.4 Special intersection cases

To tackle all possible scenarios some special cases were also studied and treated separately.

- **All vehicle making the same manoeuvre:** if going to the right the code was not modified. If all have to go straight or left, to choose which vehicle would cross first it was assumed that the road running vertical would be a principal road, so with a higher priority. Therefore, the first vehicles to pass would be Car 1 and Car 3, then Car 2 and Car 4.

Direction	Condition
Right	-
Straight	priority_1 == 1 && priority_2 == 1 && priority_4 == 1
Left	priority_1 == 0 && priority_2 == 0 && priority_4 == 0

Table 5.2: Car 3 crossing conditions with same directions

- **Vehicle not present:** when a vehicle is not chosen to appear at the intersection, its place will be left empty, so, obviously, the other vehicles will not have to give way to the non-existent car. Since the crossing conditions are governed by the presence (or non-presence) of a vehicle inside the intersection, to leave this situation unaccounted for would mean that some vehicle might never cross the intersection because it is waiting for a non-existent car to pass.

To solve this problem easily it was decided that, if a vehicle is not selected, the FSM directly enters a state where all crossing-related flags are set as if the vehicle had already passed. Therefore, when the selected vehicles arrive at the intersection, they will proceed accordingly to the road rules, when one (or more) of the four ways is empty.

- **Stopping in the middle of the intersection:** following road rules, a particular situation may appear, in which one (or more) vehicles have to start occupying the intersection and stop in the middle of the road. By stopping in the middle, the right of other vehicles can get freed, allowing them to start, and ultimately untangle the intersection.

This situation presents itself when a vehicle has to turn left, the vehicle on its left has to go straight, and it also has to give way to other vehicles as described in (Sec. 5.4.3), meaning that the vehicle on its right either goes straight or left, while the vehicle in front either goes right or straight.

This translates into a two-steps condition when exiting the state going left, where at first, if the car on the left goes straight, the state *Left_turn_45* is entered to stop in the middle. After the vehicle has stopped in the middle, its condition to resume travelling are translated as if it was turning left normally. Therefore, the standard *Left_turn* state is entered when the standard conditions of (Sec. 5.4.3) are met.

To signal the vehicle has stopped in the intersection the variable *car_x_r* is set to 0, this allows the car on the left to pass and start freeing the intersection.

The results of this manoeuvre are plotted in (Fig. 5.10).

Direction	Condition
Left_45	car_2_r == 1 && priority_2 == 1
Left	(car_4_r == 0 priority_4 == 2) && (car_1_r == 0 priority_1 == 0)

Table 5.3: Car 3 special case

In (Fig. 5.5) the entire FSM relative to Car 1 is presented. Near each state that actively moves the vehicle is present the relative Matlab function. Each condition to change state was extensively described in the chapters before.

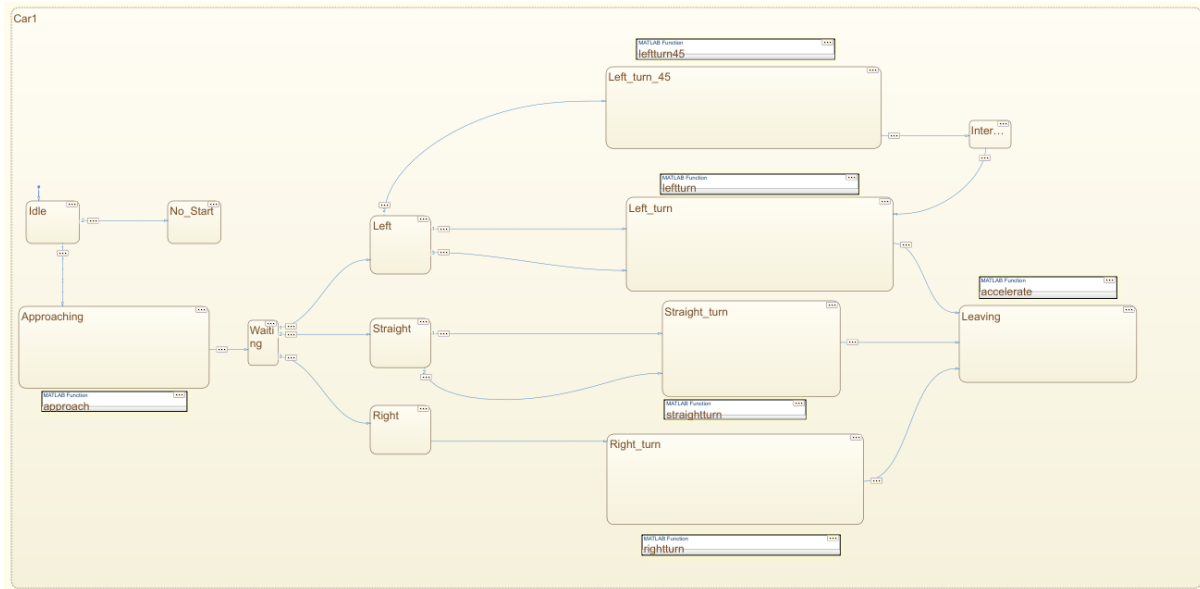


Figure 5.5: Overall FSM setup

5.5 Plotting mechanism

To show visually the vehicles' behaviour and movements it was necessary to create a top view of the intersection. As described before, the road is two-way, 12m wide, meaning the lane is actually 6m.

The main challenge consisted in converting the abscissa *pos* of each vehicle, computed in the FSM, into an (x,y) coordinate representation to be plotted in Matlab. The Matlab function `plotting_x` was created inside Simulink to tackle this problem: it transformed the abscissa into (x,y) coordinates, customized for each vehicle.

The function `plotting_x` outputs the (x,y) coordinates of the assigned vehicle, and takes as inputs:

- The abscissa *pos*.
- The direction of travel *dir_init*.
- The coordinates (x,y) computed when turning in (Eq. 5.5).
- The various flags signalling the vehicle's state *appr*, *wait*, *turn*, *leave*.

Inside the function, the vehicle's initial position is initialized in the assigned position, starting 20m from the stopping line. Subsequently a switch-case logic is used to plot accordingly to the travelled direction: inside each case a if-else logic translates the abscissa *pos* into (x,y) coordinates depending on the vehicle's state.

Using this logic, together with some tuning factors, it was possible to correctly represent how the vehicles travel along the intersection. Each dot represents the vehicle's position every 1s.

5.6 Results

In this section several plots are presented, showing all the possible combinations and casistics.

5.6.1 Different number of vehicles

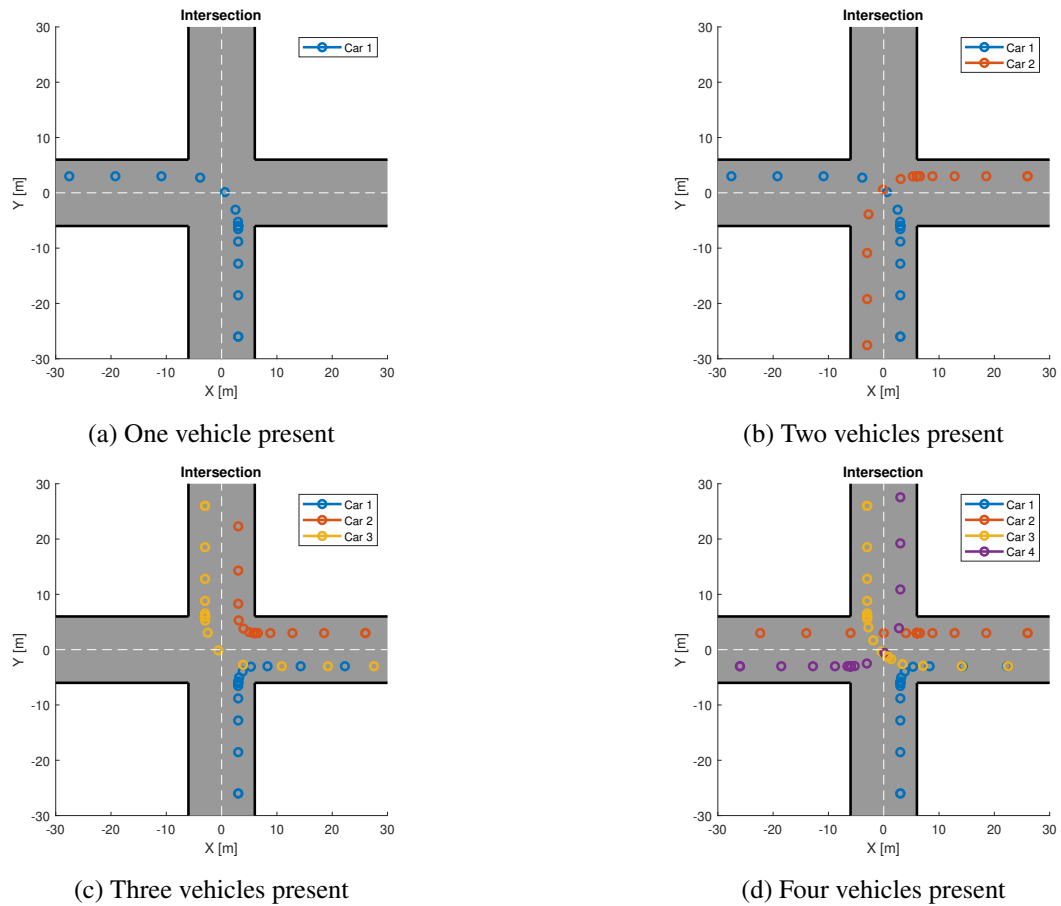


Figure 5.6: Intersection with increasing number of vehicles

The code works with just one vehicle, up to four. All combination of active/non-active cars are available, following the correct road rules, with totally random travel directions.

5.6.2 All vehicle going in the same direction

The velocity profiles show the solution of the problem arising when all the vehicles have to go either straight or left, solved by letting through Car 1 and Car 3 first (Sec. 5.4.4).

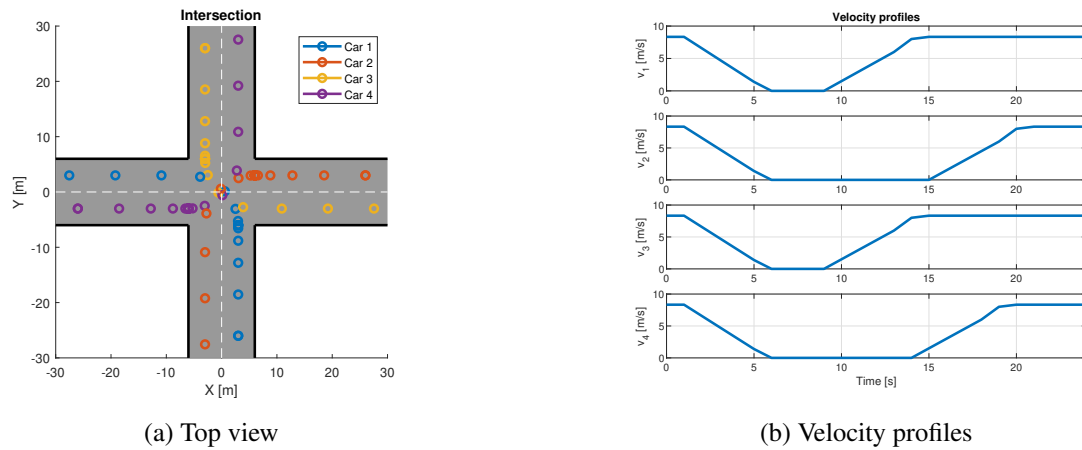


Figure 5.7: All vehicles going left

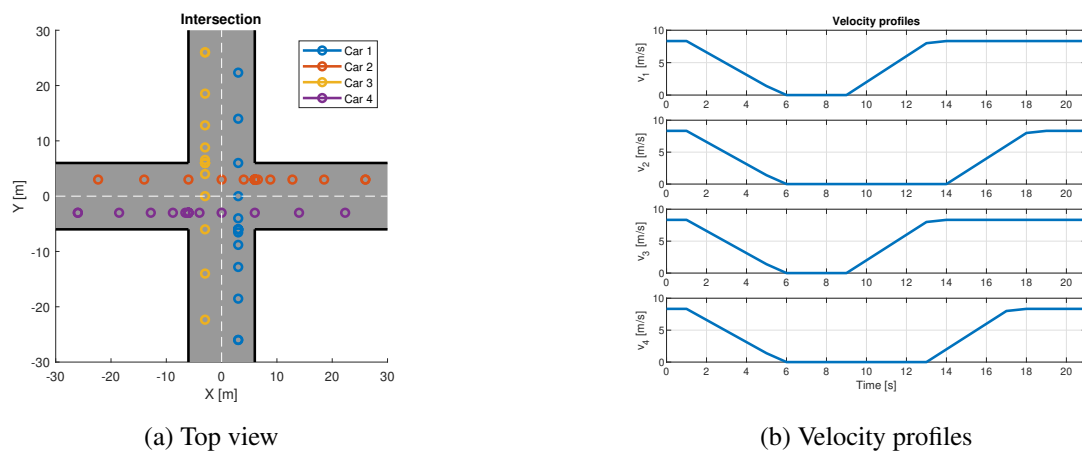


Figure 5.8: All vehicles going straight

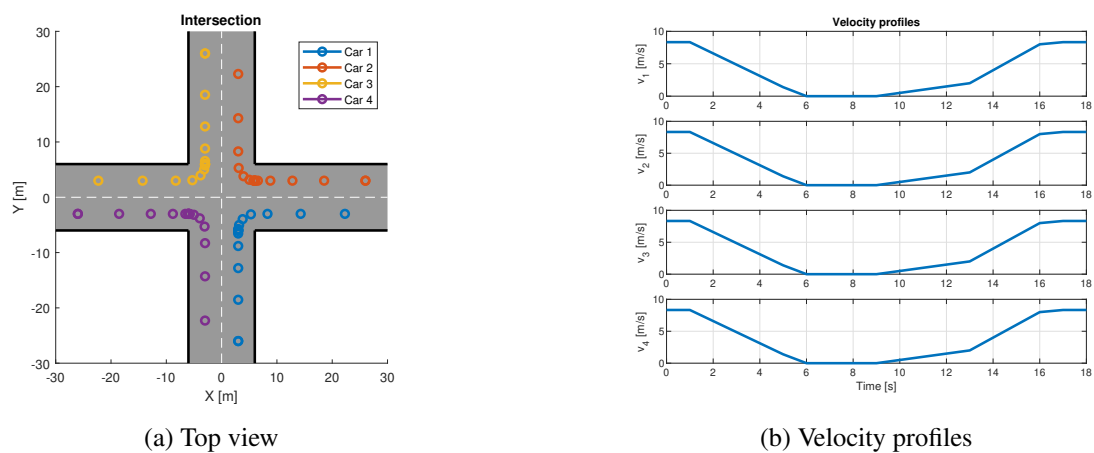


Figure 5.9: All vehicles going right

5.6.3 Stopping in the intersection

Here it is possible to see the behaviour of Car 3, which stops in the middle to let other vehicles pass, before resuming its journey, as described in (Sec. 5.4.4). In the velocity profiles the small peak between 9s and 14s shows the vehicle accelerating and then stopping in a short space, before resuming to travel as last of the four vehicles. Obviously all vehicles are able to carry out this manoeuvre, not only Car 3.

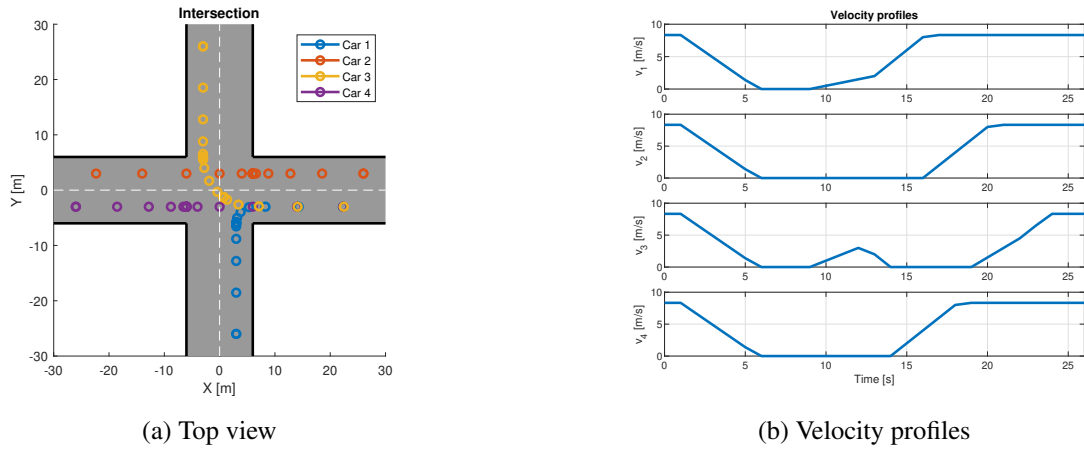


Figure 5.10: Car 3 stopping in the intersection

All the settings, conditions and states mentioned in the previous sections are visible directly in the FSM architecture.

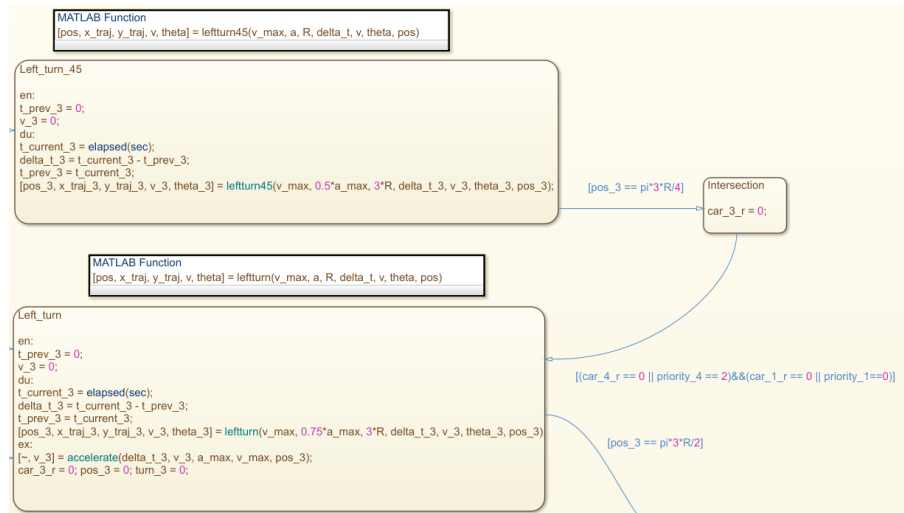
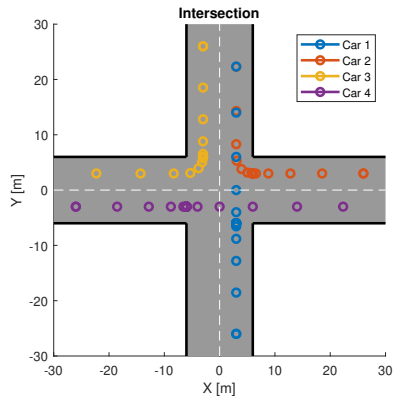


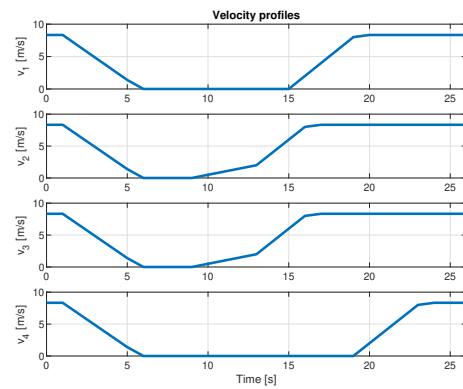
Figure 5.11: Close-up of the FSM states controlling the manoeuvre

5.6.4 Random directions

Finally some plots are presented, all run with random travel directions. These plots numeration correspond to the videos handed in.

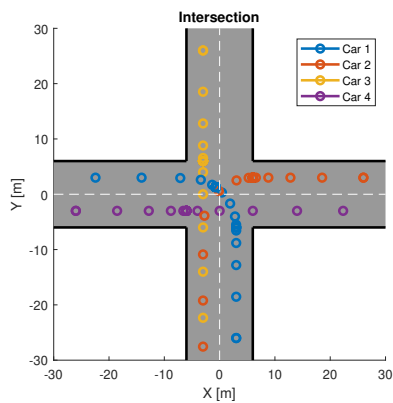


(a) Top view

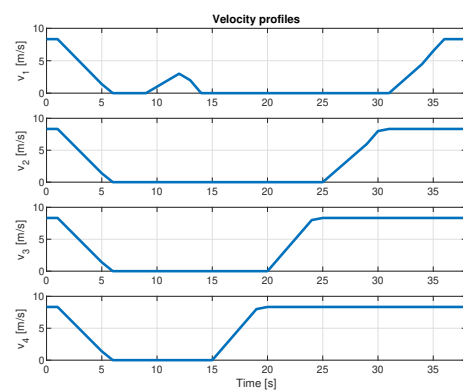


(b) Velocity profiles

Figure 5.12: Random direction 1

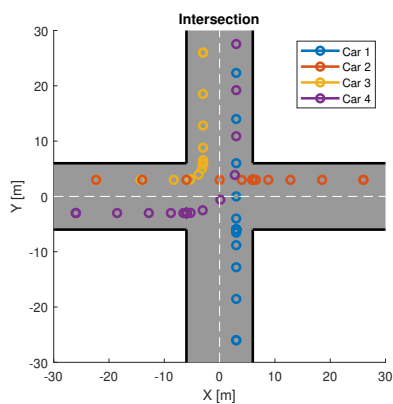


(a) Top view

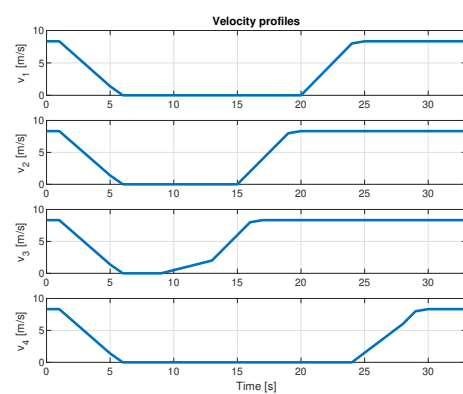


(b) Velocity profiles

Figure 5.13: Random direction 2



(a) Top view



(b) Velocity profiles

Figure 5.14: Random direction 3

5.7 Conclusions

The final model implemented allows to correctly simulate the behaviour of four vehicles passing through a four-way intersection, respecting road rules. The Finite-State Machine (FSM) architecture was able to represent all the necessary actions carried out by each vehicle: from appearing on the road, to approaching the intersection, waiting, making a manoeuvre, and finally exiting the intersection. The vehicle motion was simulated using a point mass model, with constant acceleration and deceleration profiles, and taking into account the imposed physical limits, to the likes of maximum acceleration and speed. The road rules were respected implementing a system of flags based on priorities of way depending on the chosen travel direction, allowing to manage all possible situations that may arise in real life.

Particular attention was given to special cases, like vehicles stopping inside the intersection to unlock other cars' paths, giving them way, or the absence of one or more vehicles. The parallel structure of the FSM made it possible to run several simulations by varying both the number of active vehicles and their directions, showing algorithm robustness by giving results consistent with the expectations.

Some improvements on the model can be considered. The fixed time-step of 1 second used in the FSM simulation introduces approximations in the calculation of the position and velocity, especially during acceleration or deceleration phases. A smaller or adaptive time-step could improve the precision of the simulation, resulting in more consistent velocity profiles. Additionally, the vehicle model could be refined by considering more realistic dynamics, like the influence of the turn radius on speed, or non-constant acceleration/deceleration profiles. The assumption stating the all vehicles must appear at the intersection at the same time may be removed, reworking parts of the algorithm, to simulate a more realistic situation, where rarely vehicles arrive all at the same time. Lastly, the graphical output could be improved by using smoother time-steps, enhancing trajectory realism and adding more information: for example the vehicle's current state or planned manoeuvre.

In conclusion, the implemented FSM proved to be a robust tool for simulating vehicle interaction at intersections: with its parallel structure, it is also possible to further improve and rework with the aim of representing more complex and realistic traffic environments.